What to provide so I can continue — search fix + device gate + redistribute

Date: 2026-06-23 **Author:** Claude (Opus 4.8, 1M ctx) — agent handoff to operator **Why this exists:** the search-bug fix can be *diagnosed and code-fixed* without anything from you, but **verifying it on a device and redistributing requires signing/build secrets that are not present in this environment.** This guide lists exactly what to provide, where, and in what format, plus the one diagnostic readout I still need. Follow the parts in order; PART 1 unblocks the root cause immediately.

> **§6.H SECURITY — READ FIRST.** Do **NOT** paste secret *values* into the chat. Place them as **files on disk** (they are all gitignored). The thinker login password you pasted earlier is now in the session transcript — **please rotate it.** I never stored/committed it, and I don't need it: passwordless SSH key access to thinker already works.

Status at this handoff

- **Search bug:** Phase-1 root-caused to the release **key/auth OR engine-reachability** layer — `ApiBackedTrackerClient.getString()` throwing on a non-2xx (likely **401**) or a connection failure; *not* a parse/R8 failure. The exact layer needs **one HTTP datum** (PART 1).
- **Shipped already (`922ecbca`, on both mirrors):** a §6.AC fix so per-provider search failures are recorded to Crashlytics instead of silently dropped (they were undiagnosable in the field). This does **not** fix search — it makes the failure visible.
- **Blocked:** device verification + redistribute — need the secrets in PART 2. `this environment` has **no** `.env`, `keystores/`, `app/google-services.json`, or `api-app/google-services.json`.
- **thinker:** clone done at `~/lava` (KVM live, no cgroup cap); ready to be the device gate once secrets + Android SDK are in place.

PART 1 — Immediate, needs NO secrets: the Chucker readout

This pins the search root cause so I can write the real fix.

1. **Uninstall** the release `Lava` client + `Lava API` app (clean slate).
2. From **Firebase App Distribution**, install **both `.dev` debug builds** (latest = client **1.3.10-1067** `.client.dev` + api-app **0.2.10-15** `.api.dev`). They are already distributed — no rebuild.

3. Open **`Lava API` (`.api.dev`)** → start the engine → confirm its foreground notification shows it running.
4. In the client: **fresh onboarding** → pick the on-device API → **search** (e.g. "ubuntu").
5. Open **Chucker** (its notification, or its separate launcher icon) → open the most recent **`/v1/{provider}/search`** transaction.

Report these three facts:

- [ ] **Status of `/v1/{provider}/search`** — `200` / `401` / `404` / `5xx`, **or** "no such request appears".
- [ ] Is there a **`/providers`** request, and is it **200**?
- [ ] Does search **work** on this debug pair (results render)?

> Decoder: `/providers` 200 + `/v1/...search` 401 → key/auth layer. Both fail / no request → on-device engine unreachable. Works on debug but you reported release broken → release-only (R8/signing/config).

PART 2 — Signing/build secrets: unblock device verification + redistribute

All four are **gitignored** and will never be committed. Place them as files. The canonical template for `.env` is **`.env.example`** at the repo root — copy it and fill real values.

2A — `.env` at the repo root

Copy `.env.example` → `.env`, then set at minimum (the **build-critical** subset; the Android build's auth-blob codegen reads these and the build FAILS without them):

```
KEYSTORE_PASSWORD=<real signing password>
KEYSTORE_ROOT_DIR=keystores
LAVA_AUTH_FIELD_NAME=Lava-Auth
LAVA_AUTH_CURRENT_CLIENT_NAME=<e.g. android-1.3.11-1068>
LAVA_AUTH_ACTIVE_CLIENTS=<client-name>:<uuid>[,<older>:<uuid>...]
LAVA_AUTH_OBFUSCATION_PEPPER=<base64 pepper>
LAVA_AUTH_HMAC_SECRET=<base64 hmac secret>   # API side
```

Additionally, to Firebase-redistribute (PART 3 step "ship"):

```
LAVA_FIREBASE_TOKEN=<from `firebase login:ci`>
LAVA_FIREBASE_PROJECT_ID=<project id>
LAVA_FIREBASE_ANDROID_APP_ID / _DEV_APP_ID    #
client .client/.client.dev
LAVA_FIREBASE_API_APP_ID / _DEV_APP_ID        # api-
app .api/.api.dev
LAVA_FIREBASE_TESTERS_OWNER/_DEVELOPER/_TESTER=<emails>
```

Tracker credentials (`RUTRACKER_*`, `KINOZAL_*`, …) are **only** needed for the credentialed Challenge Tests (C2/C9/C10). The search bug is on the **no-auth Internet Archive** provider, so they are **not** required to repro/verify it — provide them later if you want the full matrix.

2B — `keystores/` directory

Place the two signing keystores referenced by `app/build.gradle.kts` (same names for `api-app`):

```
keystores/debug.keystore     # keyAlias "debug",   store+key
password = KEYSTORE_PASSWORD
keystores/release.keystore   # keyAlias "release", store+key
password = KEYSTORE_PASSWORD
```

The **release** keystore MUST be the *same* one that signed the distributed 1.3.x releases (so the matched-pair signature permission grant works and existing installs upgrade).

2C — `app/google-services.json`

The client Firebase config. MUST contain the client packages **`digital.vasic.lava.client`** and **`digital.vasic.lava.client.dev`**. Without it the Google-Services Gradle plugin fails the `:app` build.

2D — `api-app/google-services.json`

The api-app Firebase config. MUST map the api-app packages **`digital.vasic.lava.api`** and **`digital.vasic.lava.api.dev`** to their **own** Firebase app ids (this file previously had the *client's* app id by mistake — Defect B, fixed 2026-06-14; double-check it maps the `.api` ids). Without it the `:api-app` build fails.

Where to place the secrets — pick one

- **Option A (recommended): this working tree** (`/Volumes/T7/Projects/lava`). This Mac already has the Android SDK + a working Gradle, so I can build the signed APKs here and transfer them to thinker for the emulator run. Drop the files at the paths above; they're gitignored.
- **Option B: on thinker** at `~/lava/` (same relative paths). Then I build everything on thinker. I will install the Android SDK on thinker myself (secrets-free) — you only provide the four secret artifacts.

> Transfer suggestion (either option): `scp`/`rsync` the files directly, or drop them in place. Do **not** email/paste secret contents.

PART 3 — What I do once each part lands

You provide	I do (autonomous, anti-bluff)
PART 1 Chucker readout	Root-cause the exact layer → write the real search fix + a falsifiability-proven unit test (no secrets needed) → push.
PART 2 secrets	Build the matched debug pair → install on the thinker **KVM emulator** → reproduce + confirm the fix on device (§6.Z evidence) → §6.AE Challenge matrix.
PART 2 + Firebase keys	§6.AA two-stage redistribute (debug → verify → release), both apps, with §6.Z device-gated evidence. §6.Y version bump applied.

I will **not** build/sign/distribute with placeholders (that is the §6.L bluff that bricked 1.2.19) — only with the real artifacts above.

PART 4 — Two operator decisions still open (not blocking PART 1)

1. **The `11f29ff8` "Auto-commit" pin sweep.** An external automation on your machine swept all 20 submodule pins to upstream HEAD (19 clean forward moves, 1 divergent: `llm_orchestrator`), bypassing the freeze-by-default rule (`constitution` is supposed to advance only via CONST-049). My recommendation: **don't mass-revert** (targets are legit), but **neutralize the auto-committer** (find the cron/launchd/IDE job emitting "Auto-commit") and **deliberately review** just the `constitution` + `llm_orchestrator` pins. Tell me how you want to proceed.
2. **§6.AB WEAK Challenge backlog (10).** Closing the 8 Tier-1 ones needs widening some feature composables `private → public` (a product-surface change you gate) + the device matrix to falsifiability-rehearse. Want me to do that as part of the device-gate work once PART 2 lands?

Security recap (§6.H)

- Provide secrets as **files**, never as chat text.
- All four artifacts are **gitignored** — I will never `git add` them (verified: `.gitignore` covers `.env`, `keystores/`, `**/google-services.json`).
- **Rotate the thinker password** you pasted earlier.
- If any real credential ends up in a tracked file, that's a §6.H incident — I'll halt and flag it.